

SOILHELP — COMPLETE PROJECT ANALYSIS REPORT

Team Reference Document | Judge Preparation Guide

Field	Details
Project	SoilHelp — AI-Powered Soil Analysis App
Platform	React Native (Expo) + Python FastAPI Backend
Region	Karnataka, India
Team Size	6 Members
Purpose	Full project understanding + Judge Q&A preparation

SECTION 1 — PROJECT OVERVIEW

SoilHelp is an AI-powered mobile application for Indian farmers. A farmer or field officer photographs a soil sample with a smartphone and receives soil type, NPK nutrients, pH, crop recommendations, and fertilizer advice — all in under 10 seconds, directly in the field.

Problem Solved:

Traditional soil testing requires sending samples to a government lab which takes 3-7 days and costs money. Small farmers in Karnataka cannot afford this delay during critical sowing seasons. SoilHelp eliminates this barrier with instant, free AI-powered analysis.

Who Uses It:

- Soil Extension Officers / Field Workers — operate the app in the field
- Farmers — receive the report via Share / WhatsApp / SMS from the officer

SECTION 2 — SYSTEM ARCHITECTURE

MOBILE APP (React Native + Expo + TypeScript)
Screens: Dashboard | Register | Collect | Track | Upload | Profile | Analysis
Charts: DonutGauge | RadarChart | PieChart | PHScale | SoilProbabilityGraph
Context: LanguageContext (English / Hindi / Kannada / Tamil / Telugu)

HTTP POST /analyze (image file + soil metadata)

PYTHON BACKEND (FastAPI)
Step 1: Google Cloud Vision API -> validate image is soil + color hint
Step 2: MobileNetV2 CNN -> Soil Type (Sandy/Clay/Loam/Black/Red) + confidence
Step 3: RF + GradientBoosting -> N(%), P(ppm), K(ppm), pH
Step 4: Recommendation Engine -> crops + fertilizer + pH advice

JSON response

RESULTS SCREEN
5 SVG Charts + Crop Cards + Fertilizer Tips + Share Button

DATA LAYER
Crop_recommendation.csv | Karnataka_Crop_Fertilizer_Reference.csv
karnataka_soil_npk.csv | npk_training_data.csv
soil_images/ (Kaggle — 750 images, 5 classes x 150 each)

SECTION 3 — TECHNOLOGY STACK

Layer	Technology	Why Used
Mobile App	React Native + Expo	One codebase: Android + iOS + Web
Navigation	Expo Router (file-based)	Folder structure = screen routing
Language	TypeScript	Type safety, compile-time checks
Charts	react-native-svg	Custom Radar, Donut, Pie, pH charts
Backend	FastAPI (Python)	Async, fast, auto API docs at /docs
AI Vision	TensorFlow + MobileNetV2	Transfer learning for soil

		classification
AI Nutrients	scikit-learn (RF + GB ensemble)	Predict N/P/K/pH from image + metadata
Image Check	Google Cloud Vision API	Verify photo is actually soil
Data	ICAR + Kaggle CSVs	Real Karnataka soil data
Multi-lang	Custom React Context (i18n)	English, Hindi, Kannada, Tamil, Telugu
Image Picker	expo-image-picker	Camera capture and gallery selection

SECTION 4 — COMPLETE USER WORKFLOW

Phase 1: Farmer Registration

- Open app -> Register tab
- Fill: Farmer Name, Mobile Number, Farm Size (acres), Village
- Validation: all fields required, phone min 10 digits
- Farmer saved locally; appears in Farmers List with name/village search

Phase 2: Soil Sample Collection

- Collect tab -> 3-step guide for proper soil sampling technique
- Tap Scan QR Code -> scan the labeled sample bag barcode
- Sample ID (e.g. #SH-2026-892) linked to the farmer profile

Phase 3: Sample Tracking

- Track tab shows timeline: Collected -> Dispatched -> Lab Analysis -> Report Ready
- Status badges: In Transit (yellow) | Completed (green)

Phase 4: AI Soil Analysis — CORE FEATURE

- Upload tab -> Take Photo (camera) OR Select from Gallery
- Tap Analyse Soil Sample
- App calls GET /health to verify backend is up (5 sec timeout)
- Sends image via POST /analyze as multipart/form-data
- Shows 'Analysing with AI...' spinner (2-5 seconds)
- Navigates to Analysis Results screen

Phase 5: Results & Reporting

- Hero card: Soil type + emoji + DonutGauge confidence %
- Radar chart: N / P / K / pH / Confidence (pentagon)
- Nutrient Pie: health distribution donut chart
- Nutrient bars: N, P, K with Optimal / Low / High badges
- pH Scale: gradient strip + needle at current value
- Soil Probability Graph: bars for all 5 soil types
- Fertilizer Advice: ICAR-calibrated for Karnataka
- Recommended Crops: Kannada names, season, water need
- Share Button: plain text report via WhatsApp/SMS/Email

SECTION 5 — TEAM MEMBER ROLES & RESPONSIBILITIES

MEMBER 1 — Dashboard, Profile & Language System

Files Owned:

- `app/(tabs)/index.tsx` — Dashboard screen
- `app/(tabs)/profile.tsx` — Settings and language selector
- `app/_layout.tsx` — Root layout wrapping entire app
- `context/LanguageContext.tsx` — Multi-language state management
- `constants/i18n.ts` — Translation strings for all 5 languages

Key Concepts:

Expo Router: Folder structure IS the navigation. `app/(tabs)/index.tsx` = Dashboard tab automatically. No manual route registration needed. `(tabs)` folder = tab bar.

LanguageContext: Wraps entire app via `LanguageProvider`. `t('key')` returns correct translation. Changing language in Profile -> context updates -> every screen re-renders instantly.

Theme system: `constants/Colors.ts` has light and dark color sets. `useColorScheme()` detects device setting. All components use `theme.primary`, `theme.background` etc. Never hard-code colors.

Dashboard widgets: 4 Widget components (Farmers, Collections, Tracking, Results). Each shows a count and navigates to its screen on tap.

Judge Questions:

Q: How does multi-language work?

A: React Context stores the active language. All text uses `t('key')` from `constants/i18n.ts`. Changing language re-renders all screens instantly.

Q: How many languages are supported?

A: 5 — English, Hindi, Kannada, Tamil, Telugu.

Q: Why is language support important?

A: Rural Karnataka farmers and officers may not know English. Local language makes the app actually usable in the field.

Q: How does dark mode work?

A: `useColorScheme()` reads the device system setting. `Colors` constant provides different hex values for light/dark. Components use theme variables and switch automatically.

MEMBER 2 — Farmer Registration & Farmer List

Files Owned:

- `app/(tabs)/registration.tsx` — Registration form with validation
- `app/farmers.tsx` — Farmers list with real-time search
- `constants/FarmerData.ts` — Local farmer data store

Key Concepts:

Form validation: Checks all 4 fields filled AND phone min 10 digits. Error via `Alert.alert()`. Runs inside `handleRegister()` function.

FarmerData module: `addFarmer()` saves a farmer. `getFarmers()` retrieves list. In-memory storage (MVP). Each farmer gets an auto-generated unique ID.

FlatList: Optimized list using virtualization — only renders visible items on screen. More efficient than `ScrollView` for long lists.

Search filter: `.filter()` on farmers array, matches name OR village, case-insensitive. Updates real-time as user types via `setSearch` state.

FAB: Green + button at bottom-right (position: absolute). Navigates to registration via `router.push('/registration')`.

Judge Questions:

Q: Is farmer data stored on a server?

A: Currently stored locally on device (in-app memory). MVP version. Production would use a secure cloud database with authentication.

Q: What validation is done?

A: All 4 fields required. Phone min 10 digits. Both checked before saving. If fail, alert shows specific error.

Q: What happens with duplicate farmers?

A: No duplicate check yet — planned for v2. The unique ID is auto-generated so duplicate names are technically allowed.

Q: How does the search work?

A: Real-time `.filter()` on the farmers array. Matches name OR village, case-insensitive. No server call — instant on device.

MEMBER 3 — Sample Collection & Sample Tracking

Files Owned:

- `app/(tabs)/collection.tsx` — Collection guide + QR scanner UI
- `app/(tabs)/tracking.tsx` — Sample tracking timeline

Key Concepts:

Collection steps: Use `t('step1Title')` etc. — automatically shown in the user's chosen language via `LanguageContext`.

QR Scanner (simulated): Tapping 'Scan QR Code' opens a full-screen Modal with a viewfinder + 3-second timer. Real QR = replace timer with `expo-barcode-scanner` camera + QR decode.

Modal component: Full-screen overlay. `visible={isScanning}` controls show/hide. `animationType='fade'`. transparent background lets content below show through.

Sample ID format: #SH-YEAR-NUMBER. In production: server-generated globally unique IDs.

Timeline icons: check-circle (green) = completed | truck (blue) = in transit | circle-o (grey) = pending/not started.

Judge Questions:

Q: Is the QR scanner real?

A: Designed UI simulation. Real camera QR reading uses expo-barcode-scanner. The UI, modal, and data linking are complete — only camera decode needs connecting.

Q: How is the sample linked to a farmer?

A: QR scan -> sample bag ID linked to farmer profile. Tracking screen shows all samples with farmer name and timeline.

Q: What are the 4 tracking stages?

A: (1) Collected (2) Dispatched to Lab (3) Lab Analysis (4) Report Ready

Q: What does the QR code represent?

A: Each bag has a unique printed QR code. Scanning it links the physical bag to the digital record — prevents mix-ups.

MEMBER 4 — Image Upload & Backend Communication

Files Owned:

- app/(tabs)/upload.tsx — Image selection and upload UI
- constants/ApiService.ts — All backend API communication

Key Concepts:

expo-image-picker: Camera + gallery. Requests device permission first. Returns URI (local file path). Images cropped 4:3 at 85% quality.

Upload state machine: idle (no image) -> picked (preview + analyse button) -> uploading (spinner, disabled) -> done (navigates to results).

Health check: GET /health before upload. 5-second AbortSignal.timeout. If no response -> shows error. Does NOT wait for full HTTP timeout.

multipart/form-data: Standard HTTP format for files + text together. Image = binary file field. Soil metadata = text form fields. FastAPI reads with File(...) and Form(...).

Web vs Native URL: Web: localhost:8000 (browser + server on same machine). Android/iOS: must use PC LAN IP (e.g. 192.168.0.9:8000). ApiService.ts checks Platform.OS === 'web' automatically.

Error handling: (1) Backend unreachable, (2) Server HTTP error, (3) Network failure. All show red error box. App never crashes.

Judge Questions:

Q: What if the server is slow?

A: Spinner shows with 'Analysing with AI...' text. 20MB image size limit prevents timeouts. Health check fails fast in 5 seconds.

Q: How does the image get sent?

A: multipart/form-data POST. Image = binary file field. Soil metadata = form text fields. All in one single request. FastAPI unpacks with File() and Form().

Q: Why health check first?

A: Better UX. Without it, user waits 30+ second HTTP timeout before seeing an error. Health check fails in 5 seconds.

Q: Why different IP for mobile vs web?

A: On mobile, localhost = the phone itself, not the developer's PC. Server runs on PC — must use PC LAN IP (run ipconfig on Windows to find it).

MEMBER 5 — Machine Learning Backend & Model Training

Files Owned:

- backend/main.py — FastAPI server, endpoints, startup lifespan
- backend/model/predict.py — CNN + NPK models + inference pipeline
- backend/train_models.py — Model training entry point
- backend/utils/vision_api.py — Google Cloud Vision API integration
- backend/data/ — CSV datasets + soil image folders

Key Concepts:

FastAPI Server: Runs at `http://localhost:8000`. GET /health = status check. POST /analyze = AI pipeline. CORS enabled for Expo web. lifespan loads models ONCE at startup. Auto docs at /docs.

Google Cloud Vision: Sends image (base64) to Google cloud. Gets labels + dominant color. Validates with soil keywords. If no API key -> mock fallback, app works fully.

CNN Soil Classifier: MobileNetV2 base (frozen) + head: GlobalAvgPooling -> Dense(256,ReLU) -> Dropout(0.4) -> Dense(128,ReLU) -> Dropout(0.3) -> Dense(5,Softmax). Phase 2: unfreeze top 30 layers. Input: 224x224 RGB. Output: 5 probabilities.

NPK Regressor: Feature vector = 210 dims: 192 (color histogram, 64 bins x 3 channels) + 5 (soil one-hot) + 5 (texture one-hot) + 8 (soil measurements). Ensemble: 60% RandomForest + 40% GradientBoosting. Outputs: N, P, K, pH.

Training data: Real Karnataka ICAR + Kaggle CSV + 3000 ICAR-calibrated synthetic samples (Beta distribution) + 150 Karnataka sub-range per soil class.

Commands: `python train_models.py` (NPK only, ~30 sec) | `python train_models.py --soil-data ./data/soil_images` (+ CNN, 10-30 min).

Judge Questions:

Q: Why MobileNetV2 instead of custom CNN?

A: Transfer learning. Pre-trained on 1.4M images — already knows edges, textures, colors relevant to soil. Only retrain final layers. Needs fewer images, trains much faster.

Q: How accurate is NPK prediction?

A: Trained on ICAR-calibrated Karnataka data. Accuracy improves when optional measurements (moisture, EC, temperature) are provided. Ensemble reduces variance vs a single model.

Q: What if no soil image dataset?

A: NPK regressor trains from ICAR synthetic data in ~30 seconds (no images needed). CNN falls back to Vision API color hints. App is fully functional.

Q: What is ICAR?

A: Indian Council of Agricultural Research — apex government body for agricultural standards. All NPK thresholds are ICAR-calibrated for Karnataka district-level conditions.

Q: What is Transfer Learning?

A: Taking a model trained on millions of images (MobileNetV2 on ImageNet) and retraining only the final layers on a small new dataset (soil photos). Pre-trained layers detect features; head classifies them as soil types.

Q: What is an Ensemble Model?

A: Combining two ML models and blending predictions. RandomForest (60%) + GradientBoosting (40%) = more accurate and stable than either alone.

MEMBER 6 — Recommendation Engine & Results Visualization

Files Owned:

- backend/utils/recommendations.py — Crop + fertilizer recommendation logic
- app/analysis.tsx — Full AI results display screen
- components/charts/ — All 5 custom SVG chart components

Key Concepts:

Crop database (2 levels): National (CROP_DB): general crops per soil type. Karnataka (STATE_CROP_DBS): crops with Kannada names e.g. 'Ragi (ರಾಗಿ)', 'Cotton / Hatti (ಹತ್ತಿ)'.

Fertilizer rules (ICAR): N<0.15%=Very Low->Urea 100kg/ha | N 0.20-0.40%=Adequate | P<10ppm=Very Low->DAP/SSP | K<100ppm=Very Low->MOP 75-100kg/ha.

Karnataka micronutrient tips: Black->Zinc Sulphate 25kg/ha | Red->Borax+FeSO4 | Sandy->slow-release urea | Clay->Gypsum 200kg/ha | Loam->FYM 5t/ha.

pH advice: pH<5.5=highly acidic->lime | 6.5-7.5=neutral=IDEAL | >8.5=highly alkaline->gypsum/sulfur.

5 Charts (react-native-svg): DonutGauge (confidence arc) | NutrientRadarChart (pentagon) | NutrientPieChart (health donut) | SoilProbabilityGraph (bars) | PHScale (gradient + needle).

NutrientBar logic: Fill % = value / (max x 1.5). Two green markers show optimal range bounds. Badge: Green=Optimal | Yellow=Low | Red=High.

Share feature: React Native Share.share() with formatted text. Device shows native share sheet (WhatsApp, SMS, Email, etc.).

Judge Questions:

Q: How are crop recommendations made?

A: Rule-based lookup. Takes soil type + state, finds crops in knowledge database (curated from ICAR and Karnataka KVK). No extra ML — reliable agricultural knowledge base.

Q: Why Kannada crop names?

A: Target users are Karnataka farmers/officers. 'Ragi' alongside 'Finger Millet' is immediately recognizable in the field without translation.

Q: How does the pH scale chart work?

A: Custom SVG gradient rectangle (red=acidic, green=neutral, blue=alkaline). Triangle needle at: (pH / 14) x chartWidth pixels from left. Optimal zone 6.0-7.5 highlighted.

Q: What does the radar chart show?

A: 5 axes: N, P, K, pH, Confidence. Each normalized to optimal max. Filled polygon connects points. Small/irregular shape = nutrient deficiencies present.

Q: How does share work?

A: React Native Share.share() with formatted text. Device shows native share sheet. User picks WhatsApp, SMS, Email, or any installed app.

SECTION 6 — DATA FLOW (END TO END)

1 Capture soil photo

SECTION 7 — KEY AGRICULTURAL & TECHNICAL CONCEPTS

N - Nitrogen: Promotes growth, chlorophyll production. Deficiency: yellowing, stunted plant. Too high: weak stems, disease risk. Optimal: 15-30 ppm. Fertilizer: DAP or SSP (Single Super Phosphate).

2 Image URI returned

P - Phosphorus: Root development, flowering, seed formation. Deficiency: purple leaf undersides. Optimal: 15-30 ppm. Fertilizer: DAP or SSP (Single Super Phosphate).

K - Potassium: Disease resistance, water regulation. Deficiency: brown leaf edges. Optimal: 140-250 ppm. Fertilizer: Muriate of Potash.

3 Build multipart request

pH: Soil acidity/alkalinity. Scale 0-14. Optimal for most Indian crops: 6.0-7.5. Fix acidic: Agricultural lime. Fix alkaline: Gypsum.

AI ANALYSIS PIPELINE

ICAR: Indian Council of Agricultural Research. Provides guidelines for agricultural standards. All NPK thresholds in SoilHelp are ICAR-calibrated for Karnataka.

Google Vision

soil check + color

MobileNetV2 CNN

soil type + confidence

RF + GB ensemble

N / P / K / pH

GradientBoosting (40%) = more accurate and stable than either alone.

Kharif Season: Monsoon crops. June-November. Rice, Cotton, Ragi, Soybean, Maize.

Rabi Season: Winter crops. December-May. Wheat, Mustard, Onion.

4 Recommendation engine

Red Laterite Soil: South KA (Bengaluru Rural, Kolar, Tumkur). Acidic, iron-rich. Best crops: Ragi, Groundnut, Red Gram.

Black Vertisol Soil: Chitradurga, Bellary. Heavy, moisture-retentive. Best crops: Cotton, Sorghum.

5 JSON response

Alluvial/Loam Soil: Hassan, Mysuru valleys. Most fertile in Karnataka. Best crops: Maize, Vegetables, Arecanut.

Sandy Soil: Chitradurga, Bellary. Low moisture retention. Best crops: Groundnut, Coconut, Cashew.

Clay Soil: Coastal/Mysuru. Heavy, moisture-retentive. Best crops: Paddy, Sugarcane, Arecanut.

7 Share report

SECTION 8 — USER QUESTIONS (WHAT YOU SHOULD KNOW)

Q: What problem does SoilHelp solve?

A: Traditional soil analysis is slow and expensive. SoilHelp provides instant AI-powered soil analysis and crop recommendations in under 10 seconds, free of charge.

Q: What is the main

Charts + nutrient status + crop cards

A: MobileNetV2 transfer learning (visual soil classification) + RF+GB ensemble (NPK prediction), both calibrated to ICAR Karnataka district-level soil survey data. Region-specific, agronomically grounded — not generic global data.

Q: How accurate is the AI analysis?

A: CNN uses MobileNetV2 with data augmentation. NPK ensemble uses ICAR-calibrated Karnataka data. Accuracy improves when optional soil measurements (moisture, EC, temperature) are provided alongside the image.

Q: Is this offline capable?

A: UI screens work offline. AI analysis needs the Python backend. Current demo: server on local PC, same WiFi. Production: deploy to cloud (AWS/GCP/Azure).

Q: Why FastAPI not Django or Flask?

A: FastAPI is fully async — 2-3x faster than Flask for I/O. Auto-generates API docs at /docs. Handles concurrent image uploads efficiently.

Q: How is farmer data privacy handled?

A: Farmer data stored locally on device (MVP). Images processed server-side but not permanently stored. Production adds encryption, JWT auth, GDPR compliance.

Q: Why focus on Karnataka?

A: Detailed ICAR district soil surveys available. Diverse soil types. Kannada crop names make results useful for local farmers. Ideal regional proving ground.

Q: What if the photo is not soil?

A: Google Vision labels image first. If no soil keywords found, backend logs warning. Analysis still runs. Results show yellow badge: 'Image may not be a soil sample'.

Q: How does multi-language work?

A: React Context stores active language. All text uses t('key') from constants/i18n.ts. User selects language in Profile -> context updates -> every screen re-renders. Supports: English, Hindi, Kannada, Tamil, Telugu.

Q: What are the current limitations?

A: (1) QR scanner simulated; (2) Farmer data in-memory; (3) CNN accuracy depends on image quality; (4) Backend on local network; (5) No user authentication.

Q: How was the model trained?

A: Kaggle Soil Image Dataset (750 images, 5 classes x 150) for CNN. ICAR Karnataka + Kaggle NPK CSVs for regressor. Plus 3000 ICAR-calibrated synthetic samples + 150 Karnataka sub-range per class.

Q: Why ensemble model for NPK?

A: Single RF overfits to color histogram features. Single GB brittle on small data. Blending 60% RF + 40% GB gives better, more stable predictions.

Q: How long does analysis take?

A: Typically 2-5 seconds. API response includes analysis_time_seconds. Models loaded once at startup — inference is fast.

Q: Can app work without Google Vision API?

A: Yes. Without API key, system uses mock fallback (valid soil assumed, Loam hint, 78% confidence). CNN and NPK run fully independently.

Q: Tech stack in one sentence?

A: React Native + Expo (TypeScript) app connecting to Python FastAPI that runs MobileNetV2 CNN for soil classification and RF+GB ensemble for NPK prediction, calibrated to ICAR Karnataka data.

SECTION 9 — PROJECT FOLDER MAP

```
SoilHelp/
|
+-- app/
|   +-- (tabs)/
|       |   +-- _layout.tsx           Tab bar config + icons
|       |   +-- index.tsx            Dashboard (Member 1)
|       |   +-- registration.tsx      Farmer Registration (Member 2)
|       |   +-- collection.tsx        Sample Collection + QR (Member 3)
|       |   +-- tracking.tsx          Sample Tracking Timeline (Member 3)
|       |   +-- upload.tsx            Image Upload + Analysis (Member 4)
|       |   +-- profile.tsx           Settings, Language (Member 1)
|       +-- _layout.tsx              Root layout
|   +-- analysis.tsx                 AI Results + Charts (Member 6)
|   +-- farmers.tsx                  Farmers List (Member 2)
|
+-- components/charts/
|   +-- DonutGauge.tsx   NutrientRadarChart.tsx   NutrientPieChart.tsx
|   +-- SoilProbabilityGraph.tsx   PHScale.tsx
|
+-- constants/
|   +-- ApiService.ts (Member 4)  Colors.ts  FarmerData.ts (Member 2)  i18n.ts
(Member 1)
|
+-- context/LanguageContext.tsx   (Member 1)
|
+-- backend/
|   +-- main.py (Member 5)          requirements.txt      train_models.py
|   +-- model/predict.py (Member 5)
|   +-- utils/vision_api.py (Member 5)  recommendations.py (Member 6)
|   +-- data/ (CSVs + soil_images/)
|   +-- saved_models/
|       +-- soil_classifier.h5  npk_regressor.pkl  npk_scaler.pkl
|
+-- docs/
|   +-- SoilHelp_Project_Report.md
|   +-- SoilHelp_Project_Report.docx <- THIS FILE
```

SECTION 10 — QUICK CHEAT SHEET FOR DEMO DAY

If asked about...	Direct Answer
What does the app do?	AI soil analysis from phone photo: NPK + soil type + crops in under 10 seconds

What AI is used?	MobileNetV2 CNN + RandomForest + GradientBoosting ensemble
What data was used?	ICAR Karnataka surveys + Kaggle soil images + NPK CSVs
Why Karnataka?	ICAR data available; diverse soils; Kannada language support for farmers
Backend tech?	Python FastAPI — deployable to cloud
App tech?	React Native + Expo TypeScript — Android, iOS, Web
Languages supported?	English, Hindi, Kannada, Tamil, Telugu
Is QR scanner real?	Simulated UI — real scanning ready via expo-barcode-scanner
Offline capable?	UI works offline; AI analysis needs server connection
Limitations?	MVP — no cloud, no auth, QR simulated, farmer data in-memory
Analysis time?	2-5 seconds end to end
Without Google Vision?	Yes — uses mock fallback, CNN and NPK work independently
Soil types?	5: Sandy, Clay, Loam, Black, Red
Nutrients predicted?	Nitrogen (N%), Phosphorus (ppm), Potassium (ppm), pH

Document prepared for SoilHelp Team — All 6 Members | Version 1.0

SoilHelp — AI-Powered Soil Analysis for Indian Farmers